

LLMs — by Azmi

This cheat sheet synthesizes the foundational mechanics of Large Language Models (LLMs) and the official best practices from industry leaders to help you extract the best possible code, analysis, and text.

1. The Core Concepts (LLM Dictionary)

Understanding how the engine works before you start driving.

Concept	Simple Explanation
Tokens	The fundamental building blocks of AI language. Think of them as syllables or pieces of words. Roughly, 100 tokens ≈ 75 words.
Temperature	The “creativity dial.” A low temperature (e.g., 0.1) makes outputs predictable, strict, and factual (great for coding). A high temperature (e.g., 0.8) makes outputs more creative and diverse (great for brainstorming).
Context Window	The AI’s short-term memory limit. It’s the maximum number of tokens (words/code) the model can process at once.
System Prompt	The overarching set of rules, constraints, or persona the AI must adopt <i>before</i> it looks at your specific user query.
Zero-Shot vs. Few-Shot	Zero-shot: Asking the AI to do a task with no examples. Few-shot: Giving the AI 2–3 examples of the input and your desired output format before asking it to do the task.
Hallucination	When the AI confidently generates false information, fake URLs, or code libraries that don’t actually exist.

2. General Principles & Clarity

How to communicate effectively to minimize confusion and maximize quality.

Strategy	Description
Be Direct and Specific	Avoid vague commands (“make it fairly short”) and fluffy descriptions. Use precise verbs and exact constraints (“Summarize in exactly 3 to 5 sentences”).

Strategy	Description
Positive Framing	Tell the AI what <i>to do</i> rather than what <i>not to do</i> . Instead of "Don't ask for a password," say "Refer the user to the help article."
Show, Don't Just Tell	Providing a few examples of your desired output format is the most reliable way to steer tone and structure. It makes programmatic parsing (like JSON extraction) much more reliable.
Assign a Role/Persona	Always start by defining who the AI is. A simple "You are an expert Python developer and database architect" instantly focuses the model's behavior.

3. Structuring Your Prompt

Organizing your context and instructions so the model processes them correctly.

Technique	Description
The 4 Pillars	Structure complex prompts using four distinct elements: Persona , Task , Context , and Format .
Use Delimiters & Tags	Never mix instructions with data. Wrap different parts of your prompt in XML tags (<code><instructions></code> , <code><context></code>) or use strict delimiters like <code>###</code> or <code>"""</code> to prevent confusion.
Context at the Top	Always place massive documents, logs, or reference code at the <i>very top</i> of the prompt. Put your specific instructions and query at the <i>very bottom</i> .
Quote Extraction	For massive codebases or documents, force the AI to quote the relevant data first (e.g., "Place relevant quotes in <code><quotes></code> tags, then provide your answer based only on those quotes") to cut through noise.
Iterative Refinement	Treat the AI like a conversational partner. If it gets something wrong, don't rewrite the initial prompt from scratch. Just reply: "Rewrite that function to use a <code>while</code> loop instead."

4. Advanced Reasoning & Control

Techniques for complex logic, math, and multi-step execution.

Technique	Description
Chain of Thought (CoT)	Add "Think step-by-step" or ask the AI to explain its reasoning before giving the final answer. This drastically improves logic capabilities.
Adaptive Thinking	For complex tasks, prompt the AI to plan and reflect inside <code><thinking></code> tags before finalizing the output. Let the model calibrate its own reasoning depth rather than setting hard token budgets.
Prompt Chaining	Instead of one massive prompt, break complex tasks down sequentially. Prompt 1: "Create an outline." Prompt 2: "Write section 1 based on the outline." Prompt 3: "Review section 1 for errors."
Stop Sequences	Define a set of characters (tokens) that, when generated, will instantly force the model to stop writing. Highly useful for strict formatting.

5. Coding & Agentic Workflows

Guidelines for autonomous coding, subagents, and deep repository work.

Technique	Description
Leading Words	For code generation, end your prompt with a leading word to nudge the model directly into writing code. (e.g., ending a prompt with <code>import</code> or <code>SELECT</code>).
Demand Action	AI models often default to suggesting. Instead of "Can you suggest changes?", strictly command, "Implement these edits to the authentication flow."
Anti-Hallucination Guardrails	Force the AI to look before it leaps: "You MUST read the file using your tools before making any claims about the codebase. Never speculate."
Combat Overengineering	Advanced models love to overengineer. Explicitly state: "Keep solutions minimal. Do not add docstrings to unchanged code, do not add unnecessary abstractions, and do not create helper scripts for simple tasks."
State Tracking	For multi-session coding tasks, instruct the AI to track its progress using structured files (e.g., <code>tests.json</code> , <code>progress.txt</code>) or by reading <code>git logs</code> when refreshing context windows.
The Safety Check	Prevent rogue agent behavior by adding a strict rule: "For destructive actions (<code>rm -rf</code> , <code>git push --force</code> , dropping tables), you must ask the user for confirmation before proceeding."

6. The Modern AI Developer Stack

AI IDEs, CLI tools, and the skills needed to orchestrate them in your local environment.

Tool / Skill	Description & Best Practice
AI IDEs (Cursor, Windsurf)	Code editors built around AI with codebase-wide awareness. Best Practice: Create a <code>.cursorrules</code> or <code>.windsurfrules</code> file in your project root. Use it to define strict system prompts for the project (e.g., "Always use functional React components, prefer Tailwind for styling, do not use classes").
CLI Coding Agents (Claude Code, Aider)	Terminal-based agents that can read local files, execute bash commands, and commit code. Best Practice: Run these in isolated git branches. Give them scoped, contained tasks rather than massive architectural refactors.
Model Context Protocol (MCP)	An open standard that allows your local AI tools to connect directly to external data sources. Best Practice: Use MCP servers to give your IDE access to your local database schema, Slack messages, or cloud deployment logs without ever leaving the editor.
Managing Context (The Repo Dump)	AI gets confused by too much noise. Best Practice: Do not paste your entire <code>node_modules</code> or thousands of files into an LLM. Use tools like <code>repomix</code> or <code>files-to-prompt</code> to bundle <i>only</i> the specific files related to the feature you are building.
Breaking the "Agent Loop"	Autonomous agents will sometimes get stuck trying the same failing fix over and over. Best Practice: Treat the agent like a junior dev. If it fails twice, interrupt the loop, read the error yourself, and provide human intuition ("The issue isn't the API route, it's the database connection string. Look at <code>.env</code> ").